

Running Qwen 3.6 35B A3B on Windows

Optimized Local LLM Deployment Guide for 8GB VRAM (RTX 5060 Ti) & 32GB RAM

The Core Concept: Qwen 3.6 35B A3B is a Mixture of Experts (MoE) model that activates a small subset of its parameters (~3B) per token. This guide configures `llama.cpp` to hold inactive layers in your 32GB system RAM while utilizing your 8GB RTX 5060 Ti VRAM for active execution, achieving surprisingly fast processing speed on consumer hardware.

Phase 1: Model Selection & Download

For a system with 32GB RAM and an 8GB VRAM card, the `Q4_K_M` or `Q5_K_M` GGUF quantizations provide the ideal balance between memory usage and logic retention.

1. Navigate to Hugging Face and search for an optimized repository like `Unsloth/Qwen3.6-35B-A3B-GGUF`.
2. Download the file matching your preference (e.g., `Qwen3.6-35B-A3B-Q5_K_M.gguf`).
3. Save the downloaded weights into a dedicated system folder, such as `C:\LLM_Models\`.

Phase 2: Environment & llama.cpp Setup

To avoid overhead from desktop wrappers, we run native `llama.cpp` for raw performance and complete parameter control.

1. Visit the official **llama.cpp GitHub Releases** page.
2. Download the pre-compiled Windows execution binaries built for CUDA (e.g., `llama-bXXXX-bin-win-cuda-cuXX.X-x64.zip`).
3. Extract the archive into an accessible folder on your local SSD, for example: `C:\llama.cpp\`.

Phase 3: The Optimized Startup Script

Create a reusable Windows batch script to initialize the server with proper memory flags.

1. Open Notepad or your preferred text editor.
2. Paste the following command configuration exactly:

```
@echo off
cd /d "C:\llama.cpp"

:: System Configuration Variables
set MODEL_PATH="C:\LLM_Models\Qwen3.6-35B-A3B-Q5_K_M.gguf"
set THREADS=8
set BATCH_SIZE=4096
```

```
set UBATCH_SIZE=2048
set CONTEXT_SIZE=120000

llama-server.exe ^
  -m %MODEL_PATH% ^
  -ngl 999 ^
  --n-cpu-moe 35 ^
  -c %CONTEXT_SIZE% ^
  -t %THREADS% ^
  -b %BATCH_SIZE% ^
  --ubatch-size %UBATCH_SIZE% ^
  --flash-attn on ^
  --cache-type-k q8_0 ^
  --cache-type-v q8_0 ^
  --mlock ^
  --port 8000 ^
  --host 0.0.0.0

pause
```

1. Save the file explicitly as `run_qwen.bat` inside the `C:\llama.cpp\` folder. (Verify it does not save with a trailing `.txt` extension).

Phase 4: Deep Dive into Optimization Flags

The flags specified in the script handle memory balancing behind the scenes:

-ngl 999 : Offloads all possible computational layers directly into the GPU's VRAM.

--n-cpu-moe 35 : Tells the engine to execute 35 of the MoE layers across the CPU/System RAM boundary, strategically shifting processing pipelines to handle the large 35B model size.

-b 4096 / --ubatch-size 2048 : Sets prompt processing (ingestion) metrics. If you see memory runtime faults under long token inputs, scale `--ubatch-size` back to `512` .

--cache-type-k/v q8_0 : Compresses key-value tokens into 8-bit precision, cutting memory allocation overhead in deep context sessions.

--mlock : Locks the weights into physical RAM, blocking Windows from paging memory chunks onto disk storage, preventing stutters.

Phase 5: Launching & Application Integration

Execute your setup by double-clicking `run_qwen.bat` . The runtime backend will initialize CUDA allocations and construct execution graphs. Once initialized, the engine exposes a local service endpoint at `http://localhost:8000` .

Because `llama-server` provides an OpenAI-compliant API format, you can connect it directly to standard front-end applications:

- **Open WebUI / AnythingLLM:** Set the service engine target to `OpenAI` , specify the host connection line as `http://localhost:8000/v1` , and provide a dummy authorization key.
- **Development Environments (Cline, Continue, Roo Code):** Choose `OpenAI Compatible` or `llama.cpp` inside the extension configuration panels and target port `8000` .